

ENGSE206

การกำหนดความต้องการและ การออกแบบทางซอฟต์แวร์

Week 02: Stakeholder, System Context, Scope และ Ethics

From Problem Brief → Shared Understanding → SRS Foundation

ผู้สอน: อาจารย์ธำนิษฐ์ เกียรติแก้ว

หน่วยที่ 1: ฐานคิดของวิศวกรรมความต้องการและความเข้าใจปัญหา

บทเรียน 1.2: Problem Framing, Stakeholder Analysis, System Context, Scope และ Ethics



สัปดาห์นี้เรียนอะไร: จาก Problem Brief ไปสู่ฐานของ SRS

ผลลัพธ์สำคัญ: รู้ว่าใครเกี่ยวข้อง ระบบอยู่ตรงไหน และยังไม่รู้อะไร

1

Problem Framing

เข้าใจปัญหา ผลกระทบ และผลลัพธ์ที่ต้องการ

2

Stakeholder Analysis

จำแนกผู้ใช้ ผู้ตัดสินใจ ผู้ดูแล และผู้ได้รับผลกระทบ

3

System Context

เห็น boundary, external entities และข้อมูลเข้า-ออก

4

Scope & Questions

กำหนด in/out scope, constraints, assumptions และ open questions

5

Ethics / Privacy / AI

คิดเรื่องข้อมูล สิทธิ ความเป็นธรรม และการใช้ AI อย่างรับผิดชอบ

ปลายทางของ Week 2

Stakeholder Map + Context Diagram + Scope Statement + Open Questions + GitHub Submission

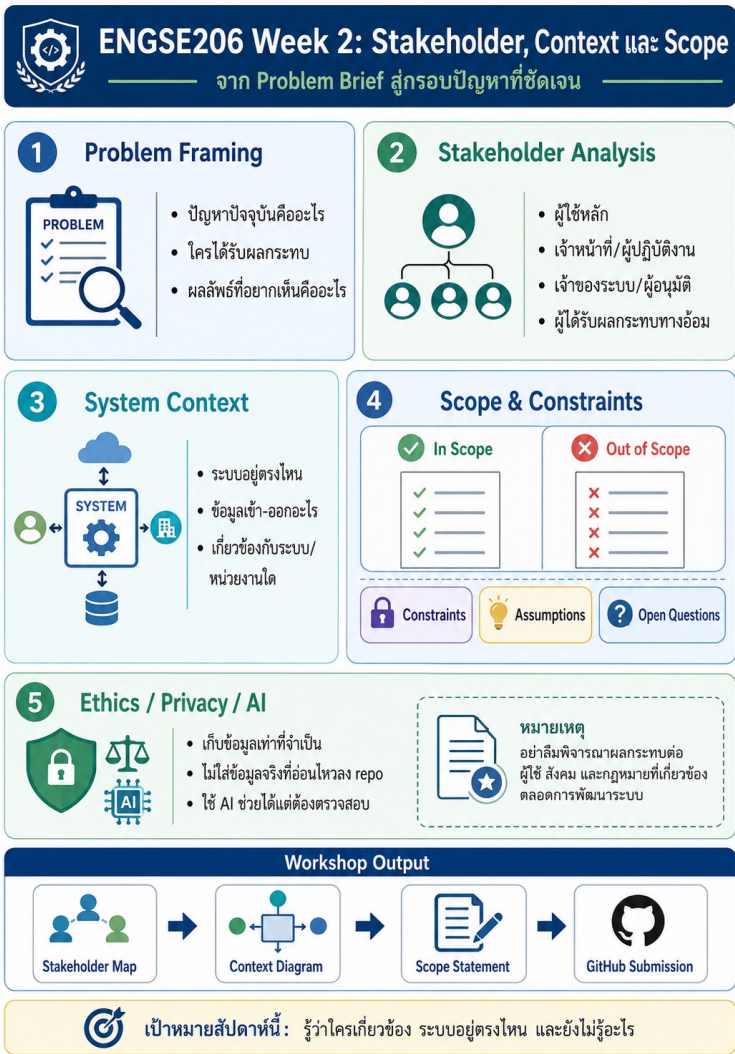


ถามเช็คความเข้าใจ

ถ้าทีมรีบเขียนฟังก์ชันใน SRS ก่อนรู้ stakeholder จะเสี่ยงอะไร?

ภาพรวมแนวคิด Week 2

จาก Problem Brief สู่กรอบปัญหาที่ชัดเจนก่อนเริ่มเขียน SRS



ทำไม SRS ไม่ควรเริ่มด้วยรายการฟังก์ชัน?

SRS ที่ดีต้องตอบได้ว่า requirement เกิดจากปัญหาใด ใครต้องการ และทีมรู้ได้อย่างไร



เริ่มจาก “ฟังก์ชัน” เร็วเกินไป

ความเสี่ยงที่เกิดขึ้น

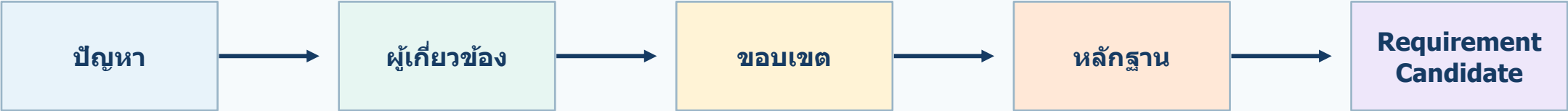
- ฟังก์ชันอาจตอบโจทย์คนละปัญหา
- ไม่รู้ว่าใครเป็นผู้ใช้จริงและใครอนุมัติ
- ขอบเขตบานปลาย เพราะยังไม่รู้ boundary
- ข้อกำหนดตรวจสอบยากและขาดหลักฐานรองรับ

หลักคิดของ Week 2
เข้าใจปัญหา + ผู้เกี่ยวข้อง + ขอบเขต + คำถามเปิด ก่อนสรุป requirement

คำถามชวนคิด: “ทำแอปจองห้อง” เป็น problem หรือ solution?

Requirement Engineering ก่อน SRS

เปลี่ยนความต้องการที่คลุมเครือให้เป็นความเข้าใจร่วมและตรวจสอบได้



ทีมพูดว่า	คำถามเชิงวิศวกรรมที่ต้องถามต่อ	เหตุผล
“ทำแอปจองห้อง”	ปัญหาการจองปัจจุบันคืออะไร? ใครได้รับผลกระทบ?	แยกปัญหาออกจาก solution
“อยากมีแรงเดือน”	ใครต้องได้รับแรง เมื่อใด และผ่านช่องทางใด?	หาเงื่อนไขก่อนเขียน requirement
“ระบบต้องใช้ง่าย”	ผู้ใช้กลุ่มใดทำงานได้บ่อยที่สุด? “ง่าย” วัดจากอะไร?	แปลงคำกว้างให้ตรวจสอบได้

สรุป: Requirement ไม่ใช่แค่ “ความสามารถของระบบ” แต่รวมเป้าหมาย กฎ ข้อจำกัด คุณภาพ และความคาดหวังของหลายฝ่าย

Problem Framing: ทำให้แน่ใจก่อนว่าแก้ปัญหาถูกเรื่อง

เริ่มจาก Current Situation → Pain Points → Stakeholders → Desired Outcome → Constraints



กรณีใกล้ตัว: ระบบจองพื้นที่ทำงานกลุ่ม

- ผู้ใช้ถามหลายช่องทาง ข้อมูลไม่รวมศูนย์
- สถานะห้องและอุปกรณ์ไม่อัปเดตพร้อมกัน
- เกิดการจองชนกันและความขัดแย้ง
- ปัญหาหลัก: ข้อมูลกระจัดกระจาย + กระบวนการไม่โปร่งใส

คำถามเช็คความเข้าใจ: “ยังไม่มีแอป” ใช่ว่าปัญหาหลักจริงหรือไม่?

คำถามช่วยทำ Problem Frame

ใช้ถามก่อนออกแบบ solution เพื่อไม่ให้ทีมหลงประเด็น

1

ปัจจุบันผู้ใช้ทำงานนี้อย่างไร?
ขั้นตอนใดทำให้เสียเวลา สับสน หรือผิดพลาด

2

ใครได้รับผลกระทบ?
ทั้งผู้ใช้โดยตรงและผู้ได้รับผลกระทบทางอ้อม

3

ถ้าไม่แก้ จะเกิดความเสี่ยงอะไร?
ต่อผู้ใช้ หน่วยงาน ข้อมูล หรือความเป็นธรรม

4

ผลลัพธ์ที่ต้องการคืออะไร?
ยังไม่รีบกำหนดวิธีแก้เชิงเทคนิค

5

มีข้อจำกัดอะไรตั้งแต่แรก?
เวลา งบ บุคลากร ความพร้อมของข้อมูล



ผลลัพธ์ที่ควรได้
Problem Frame ที่อธิบายสถานการณ์ ปัญหา
ผู้เกี่ยวข้อง ผลลัพธ์ และข้อจำกัดเริ่มต้น

นำไปใช้ต่อใน **SRS: Background, Business Need**
และ **Problem Statement**

Stakeholder Analysis: ผู้ใช้ไม่ใช่ผู้เกี่ยวข้องทั้งหมด

ระบบหนึ่งระบบมีหลายฝ่าย เป้าหมายอาจต่างกันหรือขัดแย้งกัน



Primary User

นักศึกษาที่ต้องการจองพื้นที่ — เล่ากระบวนการจริง ปัญหา และความเร่งด่วน

Operational Staff

เจ้าหน้าที่ห้อง/ผู้ดูแลพื้นที่ — ให้อำนาจอนุมัติ ขั้นตอนยืนยัน และข้อมูลที่ต้องบันทึก

System Owner / Decision Maker

หัวหน้าหน่วยงานหรือผู้อนุมัติ — กำหนดนโยบาย ข้อจำกัด และเกณฑ์สำเร็จ

Administrator / IT

ผู้ดูแลบัญชีและระบบ — ดูแลสิทธิ์ ข้อมูล การเชื่อมต่อ และความปลอดภัย

Indirect Stakeholder

ผู้ใช้พื้นที่ร่วม — สะท้อนความเป็นธรรมและผลกระทบทางอ้อม



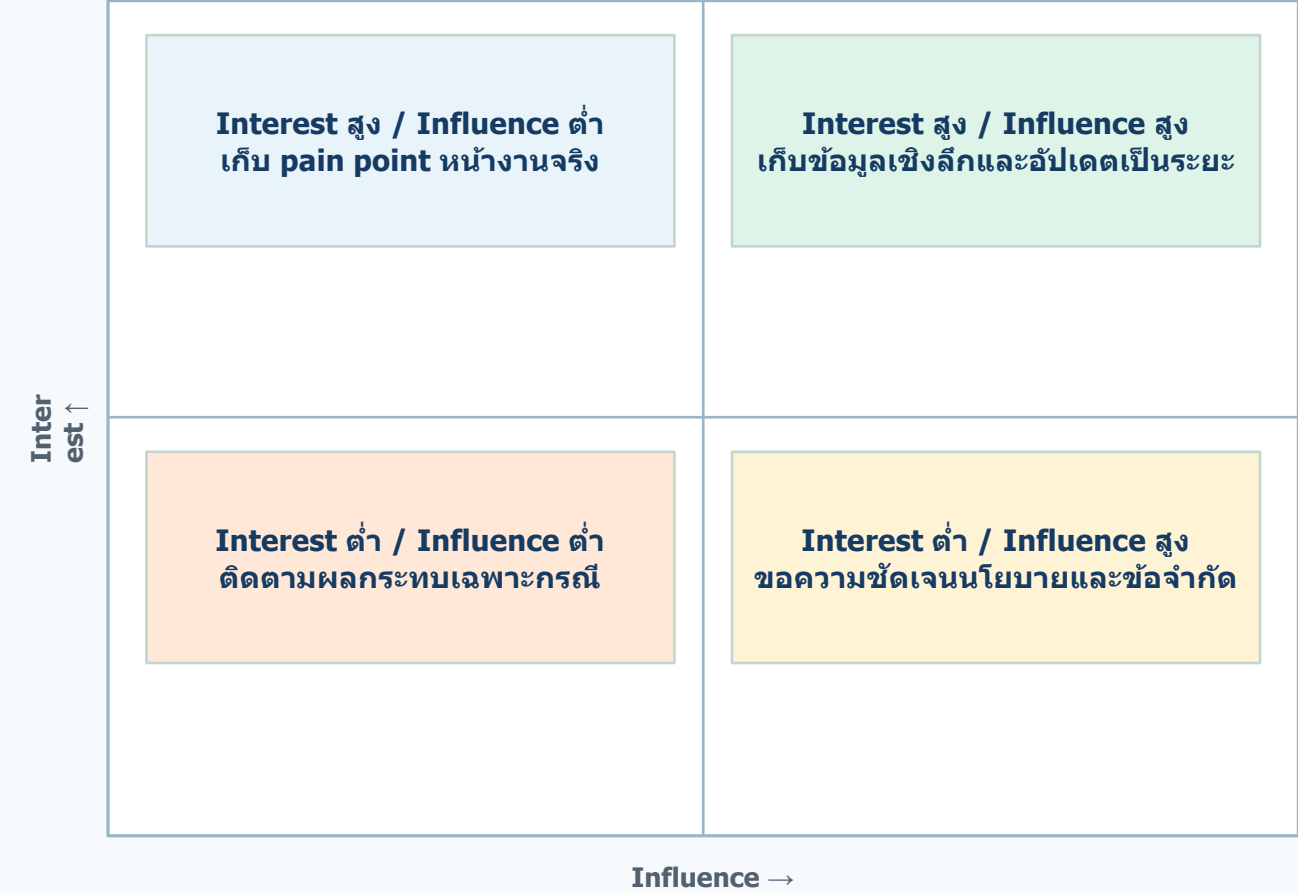
สิ่งที่ต้องเรียนรู้จากแต่ละกลุ่ม

- เป้าหมายและความต้องการ
- ข้อกังวลและความเสี่ยง
- อิทธิพลต่อการตัดสินใจ
- ข้อมูลที่ต้องให้/ต้องรับ

ถามเพื่อความเข้าใจ: ในระบบจองพื้นที่ “นักศึกษา + อาจารย์” เพียงพอไหม? ใครหายไป?

Interest–Influence Matrix

ใช้จัดลำดับการเก็บข้อมูลและวิธีสื่อสารกับ stakeholder

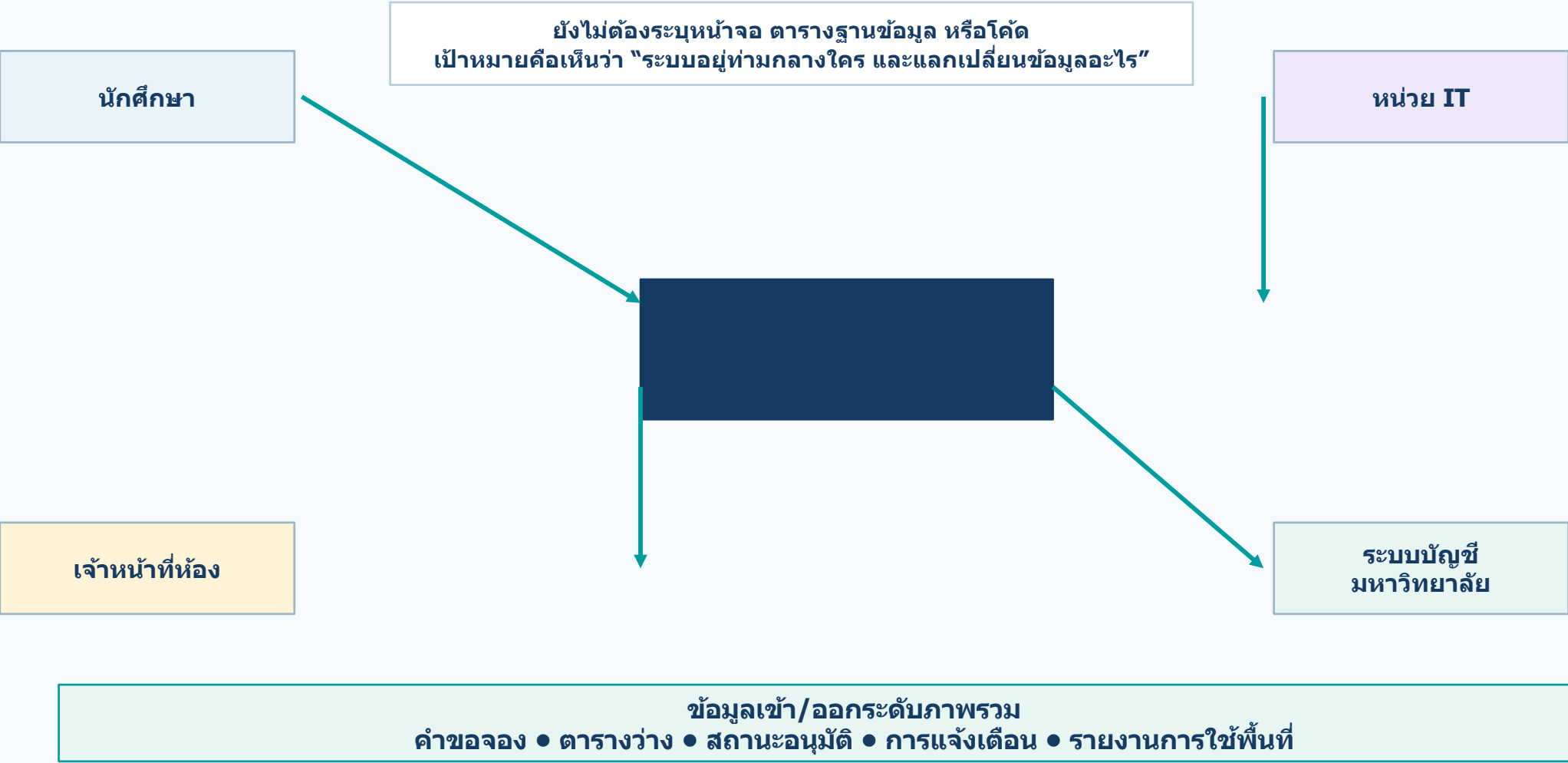


ตัวอย่างในระบบจองพื้นที่

- เจ้าหน้าที่ห้อง: interest สูง เพราะรู้ขั้นตอนจริง
- หัวหน้าหน่วยงาน: influence สูงต่อกฎและรายงาน
- นักศึกษาผู้ใช้จริง: pain point และความคาดหวัง
- IT: สิทธิ์ การเชื่อมต่อ และความปลอดภัย

System Context: มองระบบในโลกจริงก่อนลงรายละเอียด

Context Diagram ช่วยให้ boundary ชัด และลดความสับสนเรื่องสิ่งที่ระบบรับผิดชอบ



Scope, Constraints, Assumptions และ Open Questions

เครื่องมือควบคุมความเสี่ยงก่อนเริ่มเขียน SRS

In Scope

เวอร์ชันนี้ช่วยให้ใครทำอะไรได้

ตัวอย่าง: ดูสถานะพื้นที่, ส่งคำขอ, ยกเลิก, รับสถานะ

Out of Scope

เรื่องสำคัญที่ยังไม่ทำในโครงการนี้

ตัวอย่าง: ชำระเงิน, จัดซื้อ, เชื่อม ERP, พยากรณ์ขั้นสูง

Constraints

ข้อจำกัดจากเวลา นโยบาย เทคโนโลยี หรือทรัพยากร

ตัวอย่าง: ทำใน 1 เดือน, ใช้ข้อมูลจำลอง, ไม่ใช่ข้อมูลส่วนบุคคลจริง

Assumptions

สิ่งที่ทีมเชื่อว่าเป็นจริง แต่ยังไม่มีหลักฐาน

ตัวอย่าง: ผู้ใช้มีบัญชีมหาวิทยาลัย, ห้องมีรหัสชัดเจน

Open Questions

เรื่องที่ต้องถามเพิ่ม เพราะถ้าตอบผิด requirement จะผิด

ตัวอย่าง: ต้องอนุมัติไหม? จองล่วงหน้าได้กี่วัน? ใครยกเลิกแทนได้?

คำถามเช็คความเข้าใจ

“ผู้ใช้ทุกคนมีบัญชีมหาวิทยาลัย” ควรจัดเป็น Fact, Assumption หรือ Open Question?

Ethics, Privacy, Security และ Responsible AI

คุณภาพของระบบไม่ได้วัดจากฟังก์ชันเพียงอย่างเดียว



Data minimization

เก็บเฉพาะข้อมูลที่จำเป็นต่อวัตถุประสงค์

Role-based access

แต่ละบทบาทเห็น/แก้ไขข้อมูลเท่าที่จำเป็น

Transparency

ผู้ใช้เข้าใจว่าสถานะเกิดจากใคร และข้อมูลใช้เพื่ออะไร

Fairness

กฎไม่ควรเอาเปรียบผู้ใช้บางกลุ่มโดยไม่มีเหตุผล

Responsible AI

AI ช่วยร่าง/สรุปได้ แต่ต้องไม่แต่งข้อเท็จจริงแทน stakeholder



ตัวอย่างชวนคิด

ระบบนัดอาจารย์อาจเก็บเหตุการณ์การขอพบ ซึ่งอาจเป็นข้อมูลส่วนตัว

คำถาม: จำเป็นต้องเก็บละเอียดแค่ไหน? ใครควรเห็นได้?

เชื่อมกับ SRS: confidentiality • access control • auditability • usability

Artefact ของ Week 2 ส่งต่อไปเป็นส่วนใดของ SRS?

เอกสารไม่ใช่เพื่อให้ครบไฟล์ แต่เป็นฐานเหตุผลให้ requirement ทุกข้อ

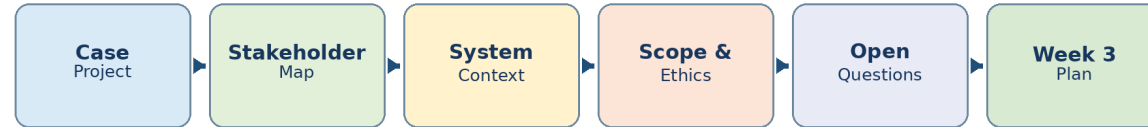
Artefact Week 2	ความหมายเชิงวิศวกรรม	ใช้ต่อใน SRS / Week 3
Problem Frame	อธิบายปัญหา เป้าหมาย และผลกระทบ	Background, Business Need, System Objective
Stakeholder Map/Profile	ระบุบทบาท เป้าหมาย และข้อกังวล	User Classes, User Characteristics, Stakeholder Needs
System Context Diagram	กำหนด boundary และความสัมพันธ์ภายนอก	System Context, Interfaces, Actors
Scope Statement	ควบคุมสิ่งที่จะทำและไม่ทำ	Scope, Out of Scope, Release Boundary
Open Questions	สิ่งที่ต้องเก็บ evidence เพิ่ม	Elicitation Plan, Interview Guide, Issue Log

หลักการง่าย: Requirement ที่ดีควรตอบได้ว่า “เกิดจากปัญหาใด ใครต้องการ และทีมรู้ได้อย่างไร”

Workshop Week 2: จาก Case Project ไปสู่ Elicitation Readiness

ทำงานกลุ่ม 3–4 คน ใช้ repository บันทึก artefact และรับ feedback แบบ Tabletop Studio

Week 2 Workflow: From Case Project to Elicitation Readiness



1. เตรียมโจทย์
ทบทวน Problem Brief และ Case Card

2. วิเคราะห์ Stakeholder
interest, influence, concern

3. วาด Context
system boundary และ data flow

4. กำหนด Scope
in/out scope, assumptions, constraints, open
questions

5. Tabletop Studio
pitch 5 นาที รับ feedback และแก้ไข

6. ส่งงาน
commit, push และบันทึก worklog

จุดเน้น: ไม่ใช่ทำให้สวยก่อน แต่ทำให้ “เหตุผลชัด + หลักฐานชัด + คำถามต่อไปชัด”

หลักการทำงานและการส่งงานผ่าน GitHub

Repository ทำให้เห็น revision history และ traceability ของ artefact

docs/02-stakeholder-context-scope.md

บันทึก stakeholder, context, scope และคำถามเปิด

diagrams/context/system-context.drawio + .png

ไฟล์ diagram และภาพสำหรับตรวจ

feedback/week-02-peer-feedback.md

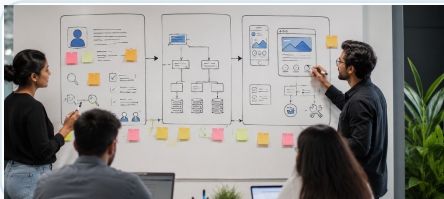
ข้อเสนอแนะจากเพื่อนและสิ่งที่ปรับแก้

submissions/week-02-submission.md

สรุปงานส่งและลิงก์ artefact

worklog

ใครทำอะไร เมื่อใด และหลักฐานใด



คำถามก่อนส่ง

1) ขอบเขตชัดเจน? 2) Open Questions นำไปเก็บข้อมูลได้จริงไหม? 3) ใช้ AI อย่างรับผิดชอบไหม?

แนวคำถามสำหรับนำเสนอ 5 นาที

ใช้คำถามเหล่านี้ช่วยให้ทีมและเพื่อนให้ feedback ได้ตรงจุด

1

ปัญหาปัจจุบันและผลกระทบที่กลุ่มเข้าใจคืออะไร?

2

Stakeholder ใดสำคัญที่สุด และเหตุใด?

3

ขอบเขตของระบบอยู่ตรงไหน? เรื่องใดเป็น out of scope?

4

Open Question ที่เสี่ยงที่สุดคืออะไร และจะนำไปทำอะไรต่อใน Week 3?



Feedback ที่ดีควรช่วยให้ทีมรู้ว่า
"ต้องถามใครเพิ่ม" "ต้องหาหลักฐานอะไร" และ
"ต้องจำกัดขอบเขตตรงไหน"

ถามเช็คความเข้าใจ: ถ้าคำถามเปิดตอบผิด จะทำให้ requirement ส่วนใดผิดได้บ้าง?

Self-check ก่อนจบสัปดาห์

ใช้ประเมินว่างานของทีมพร้อมส่งต่อไป Week 3 หรือยัง

☐ อธิบายความแตกต่างระหว่าง problem และ solution ได้

☐ Context Diagram แสดง boundary และข้อมูลเข้า-ออกชัดเจน

☐ Open Questions นำไปสร้าง Elicitation Plan ได้

☐ ระบุ stakeholder ได้มากกว่าผู้ใช้หลักเพียงกลุ่มเดียว

☐ In Scope / Out of Scope มีเหตุผลและไม่กว้างเกินเวลาเรียน

☐ หลีกเลี่ยงข้อมูลส่วนบุคคลจริง และตรวจการใช้ AI อย่างรับผิดชอบ

เป้าหมายสุดท้ายของ Week 2
รู้ว่า “ใครเกี่ยวข้อง” “ระบบอยู่ตรงไหน” “ทำ/ไม่ทำอะไร” และ “ยังต้องถามอะไรต่อ”

สรุป: จาก Problem Brief สู่แผนเก็บ Requirement

Week 2 สร้างฐานของ SRS และส่งต่อ Open Questions ไปยัง Week 3



สิ่งที่นักศึกษาควรจำ

- Requirement ที่ดีต้องมีเหตุผลและหลักฐาน
- Stakeholder หลายกลุ่มทำให้ระบบสมดุลขึ้น
- Context และ Scope ช่วยป้องกันงานบานปลาย
- Open Questions คือวัตถุดิบของการสัมภาษณ์/เก็บข้อมูลใน Week 3

คำถามปิดท้าย

ถ้ากลุ่มของคุณต้องเลือก Open Question ที่สำคัญที่สุด 1 ข้อ จะเลือกข้อใด และเพราะอะไร?